

P00 en pratique



Sidi Mohamed Ahmed M'bareck

Sommaire

- Une classe
- un objet
- Constructeur
- Visibilité
- Héritage



Sidi Mohamed Ahmed M'bareck

Qu'est-ce qu'une classe ?

- Les classes sont des moules, des patrons qui permettent de créer des objets en série sur le même modèle. On peut se représenter une classe comme le schéma de construction ainsi que la liste des fonctionnalités d'un ensemble d'objets.
- Une classe contient des propriétés et des méthodes :
 - ☐ Propriété = variable de classe
 - ☐ Méthode = fonction de classe



Exemple de classe

Class client{

private \$id;
private \$nom;
private \$prenom;

Propriétés

Public function afficherClient(){
//...
}

Méthodes

Public function ModifierClient(){
//...
}



Qu'est-ce qu'un objet ?

- Un objet est une instance de classe.
- Avec une classe, on peut créer autant d'objets qu'on veut.
- Une classe est une déclaration, alors qu'un objet est contenu dans une variable, créé à partir de notre classe avec le mot clé « new ».
- **Appeler les méthodes de l'objet**
- Pour appeler une méthode d'un objet, il va falloir utiliser un opérateur : il s'agit de l'opérateur -> (une flèche composée d'un tiret suivi d'un chevron fermant).



Exemple de création d'objets

```
// Une classe Task
class Task
{
    public $title;
    public $description;
}

// Un objet "Task".
$task1 = new Task();
// Un deuxième objet "Task".
$task2 = new Task();

// Accès à un objet
$task1->title = "Comprendre la POO";
$task1->description = "La POO est magnifique car ...";
```

The logo of the National Union of Students (U.N.E.M.) is a circular emblem. It features a green map of Mauritania in the center, with a white hand holding a pen writing on it. Below the map is an open book. The text "الاتحاد الوطني لطلبة موريتانيا" (National Union of Students of Mauritania) is written in Arabic script along the top inner edge of the circle, and "U.N.E.M" is written in green capital letters along the bottom inner edge.

Exercice 1: Création d'une classe

- Créez un fichier 01_Salle_creation.php
- Dans ce fichier, créez une classe 'Salle' (avec un S majuscule) avec les propriétés suivantes
 - \$longueur
 - \$largeur
- Créez deux objets 'Salle' à partir de cette classe
 - Remplissez leurs propriétés (\$longueur et \$largeur)
 - Affichez-les avec la fonction print_r()



- <?php
- class Salle{
- public \$longueur;
- public \$largeur;



- `$salle1= new Salle();`
- `$salle2= new Salle();`
- `$salle1->longueur=8;`
- `$salle1->largeur=5;`
- `$salle2->longueur=20;`
- `$salle2->largeur=10;`



`print_r($salle1);`

Le constructeur

- Comment marche un constructeur ?
 - Le constructeur est une méthode dite "magique" : elle est appelée automatiquement.
 - Le constructeur est appelé lors de l'instanciation d'une classe, c'est à dire la création d'un objet à partir d'une classe.
- A quoi ça sert ?
 - On appelle souvent le constructeur pour charger des propriétés (par exemple une connexion à la base de données).



Exercice 2 Constructeur

- Créez un fichier 04_Salle_constructeur.php
- Dans ce fichier, créez une classe 'Salle' (S majuscule) avec les propriétés \$longueur et \$largeur
- Créez un constructeur qui prend en paramètre deux variables \$long et \$larg
 - transférez ces valeurs sur les propriétés \$longueur et \$largeur.
 - Créez deux objets de type 'Salle' à partir de cette classe
 - Remplissez leurs propriétés (\$longueur et \$largeur) au moment de les créer
 - Affichez-les avec la fonction print_r()
 - IG



- <?php
- class Salle{
- public \$longueur;
- public \$largeur;
- function __construct(\$longueur,\$largeur){
- \$this->longueur=\$longueur;
- \$this->largeur=\$largeur;
- }



- `$salle1 = new Salle(8,5);`
`$salle2 = new Salle(20,15);`

- `print_r($salle1);`

`print_r($salle2);`



Un exemple pratique

- Prenons l'exemple suivant qui comprendra quelque rappels sur des notions essentielles de l'objet et nous l'expliquons au fur et à mesure d'avancer dans le cours
- un jeu de lute entre des animaux



Un exemple vaut mieux qu'un beau discours

- Nous créerons des objets "Animal"
- Nous verrons comment créer des classes
- Comment gérer les fonctions
- La connexion à la base de données
- Hydratation des objets
- Ajout, suppression modification, et affichages des données et d'autres paradigmes de l'objet



Création des objets

- On suppose que nous avons une classe Animal
- `$animalO=new Animal;`
- Ainsi, `$animalO` sera un objet de type Animal.
On dit que l'on *instancie* la classe Animal, que l'on crée une instance de la classe Animal.



Hypothèse

- Supposons qu'on un Animal destiné à participer à des combats
 - Il a une puissance
 - Il cumule de l'expérience après chaque combat (incrémentation par exemple)
 - Il subit des dommages après les combats
 - Les dommages=les dommages initiales +la puissance de l'animal qui vient de taper



Exemple un jeu de combat

- <?php
- class Animal{
- private \$dommages; // Les dégâts d'un Animal.
- private \$experience; // L'expérience d'unAnimal.
- private \$puissance;
- public function taper(\$animalOATaper){
- \$animalOATaper->dommages += \$this->puissance;
- }
- public function gagnerExperience() {
- \$this->experience= \$this->experience+ 1;



- Maintenant, nous pouvons créer une sorte de petite mise en scène qui fait interagir nos Animaux. Par exemple, nous pouvons créer un script qui fait lutter les Animaux. Le Animal 1 taperait le Animal 2 puis gagnerait de l'expérience, puis le Animal 2 taperait le Animal 1 et gagnerait de l'expérience. Procédez étape par étape :
 - créez deux Animaux ;
 - faites taper l'Animal 1 ;
 - faites gagner de l'expérience à : Animal 1 ;
 - faites taper l'Animal 2 ;
 - faites gagner de l'expérience à : Animal 2.



- Réalisez l'exercice



- <?php
- \$animalO1 = new *Animal*; // Un premier Animal
- \$animalO2 = new *Animal*; // Un second Animal
-
- \$animalO1->taper(\$animalO2); // \$animalO1 frappe \$animalO2
- \$animalO1->gagnerExperience(); // \$animalO1 gagne de l'expérience
-
- \$animalO2->taper(\$animalO1); // \$animalO2 frappe \$animalO1
- \$animalO2->gagnerExperience(); // \$animalO2 gagne de l'expérience



Les getters et les setters

- Comme vous le savez, nous ne pouvons pas accéder directement aux attributs de notre objet puisqu'ils sont privés : seule la classe peut les lire et les modifier. Par conséquent, si vous voulez récupérer un attribut, il va falloir le demander à la classe, de même si vous voulez les modifier.
- **Accéder à un attribut : l'accesseur**
- À votre avis, comment peut-on faire pour récupérer la valeur d'un attribut ? La solution est simple : nous allons implémenter des méthodes dont le seul rôle sera de nous donner l'attribut qu'on leur demande ! Ces méthodes ont un nom bien spécial : ce sont des **accesseurs** (ou *getters*). Par convention, ces méthodes portent le même nom que l'attribut dont elles renvoient la valeur. Par exemple, voici la liste des getters de notre classe Animal:



- <?php
- class Animal {
- private \$puissance;
- private \$experience;
- private \$dommages;
- public function taper(Animal \$animalOATaper){
- \$animalOATaper->dommages += \$this->puissance;
- }
- public function gagnerExperience() {
- // On aurait aussi pu écrire « \$this->experience+= 1 »
- \$this->experience++;



- // Ceci est la méthode dommages() : elle se charge de renvoyer le contenu de l'attribut \$dommages.
- public function dommages() {
- return \$this->dommages; }
-
- // Ceci est la méthode puissance() : elle se charge de renvoyer le contenu de l'attribut \$puissance.
- public function puissance() {
- return \$this->puissance;
- }
- // Ceci est la méthode experience() : elle se charge de renvoyer le contenu de l'attribut \$experience.
- public function experience() {
- return \$this->experience;



- Maintenant, comment cela se passe-t-il si vous voulez modifier un attribut ? Encore une fois, il va falloir que vous demandiez à la classe de le faire pour vous. Comme vous l'aurez peut-être deviné, ce sera par le biais de méthodes que l'on demandera à notre classe de modifier tel attribut.
- La classe doit impérativement contrôler la valeur afin d'assurer son intégrité car, si elle ne le fait pas, on pourra passer n'importe quelle valeur à la classe! Ces méthodes ont aussi un nom spécial : il s'agit de **setters** (ou *setters*). Ces méthodes sont de la forme `setNomDeLAttribut()`. Voici la liste des setters (ajoutée à la liste des accesseurs) de notre classe `Animal`:



Contrainte d'intégrité : Fixons les règles du jeu

- Puissance doit être un entier
- Entre 1 et 100



- Jetons un coup d'œil sur ce que cela donne
- Cours4/animal1.php



L'auto-chargement de classes

Pour une question d'organisation, il vaut mieux créer un fichier par classe. Vous appelez votre fichier comme bon vous semble et placez votre classe dedans. Pour ma part, mes fichiers sont toujours appelés « MaClasse.php ». Ainsi, si je veux pouvoir utiliser la classe MaClasse, je n'aurais qu'à inclure ce fichier :

- `<?php`
- `require 'MaClasse.php'; // J'inclus la classe.`



`Objet = new MaClasse; // Puis, seulement après, je me sers de ma classe.`

- Maintenant, imaginons que vous ayez plusieurs dizaines de classes... Pas très pratique de les inclure une par une ! Vous vous retrouverez avec des dizaines d'inclusions, certaines pouvant même être inutile si vous ne vous servez pas de toutes vos classes. Et c'est là qu'intervient l'auto-chargement des classes. Vous pouvez créer dans votre fichier principal (c'est-à-dire celui où vous créez une instance de votre classe) une ou plusieurs fonction(s) qui tenteront de charger le fichier déclarant la classe. Dans la plupart des cas, une seule fonction suffit. Ces fonctions doivent accepter un paramètre, c'est le nom de la classe qu'on doit tenter de charger. Par exemple, voici une fonction qui aura pour rôle de charger les classes :



- <?php
- function chargerClasse(\$classe)
- {
- require \$classe . '.php'; // On inclut la classe correspondante au paramètre passé.
- }

Essayons maintenant de créer un objet pour voir si il sera chargé automatiquement (je prends pour exemple la classe Animal et prends en compte le fait qu'un fichier Animal.php existe).



- Erreur fatale ! La classe n'a pas été trouvée, elle n'a donc pas été chargée... Normal quand on y réfléchit ! PHP ne sait pas qu'il doit appeler cette fonction lorsqu'on essaye d'instancier une classe non déclarée. On va donc utiliser la fonction `spl_autoload_register` en spécifiant en premier paramètre le nom de la fonction à charger :



- Prenons le fichier animal3
- Nommons ce fichier « Animal.php »
- Créez un fichier test
- Et placez
`spl_autoload_register('chargerClasse');`

On enregistre la fonction en autoload pour qu'elle
soit appelée dès qu'on instanciera une classe non
déclarée.



- Exécutez la classe test pour voir ce que cela donne



La visibilité

- Privé (private)
 - N'est accessible que dans la classe courante.
- Protégé (protected)
 - Est accessible dans la classe courante et dans les classes dérivées.
- Publique (public)
 - Est accessible partout, même à l'extérieur de la classe.

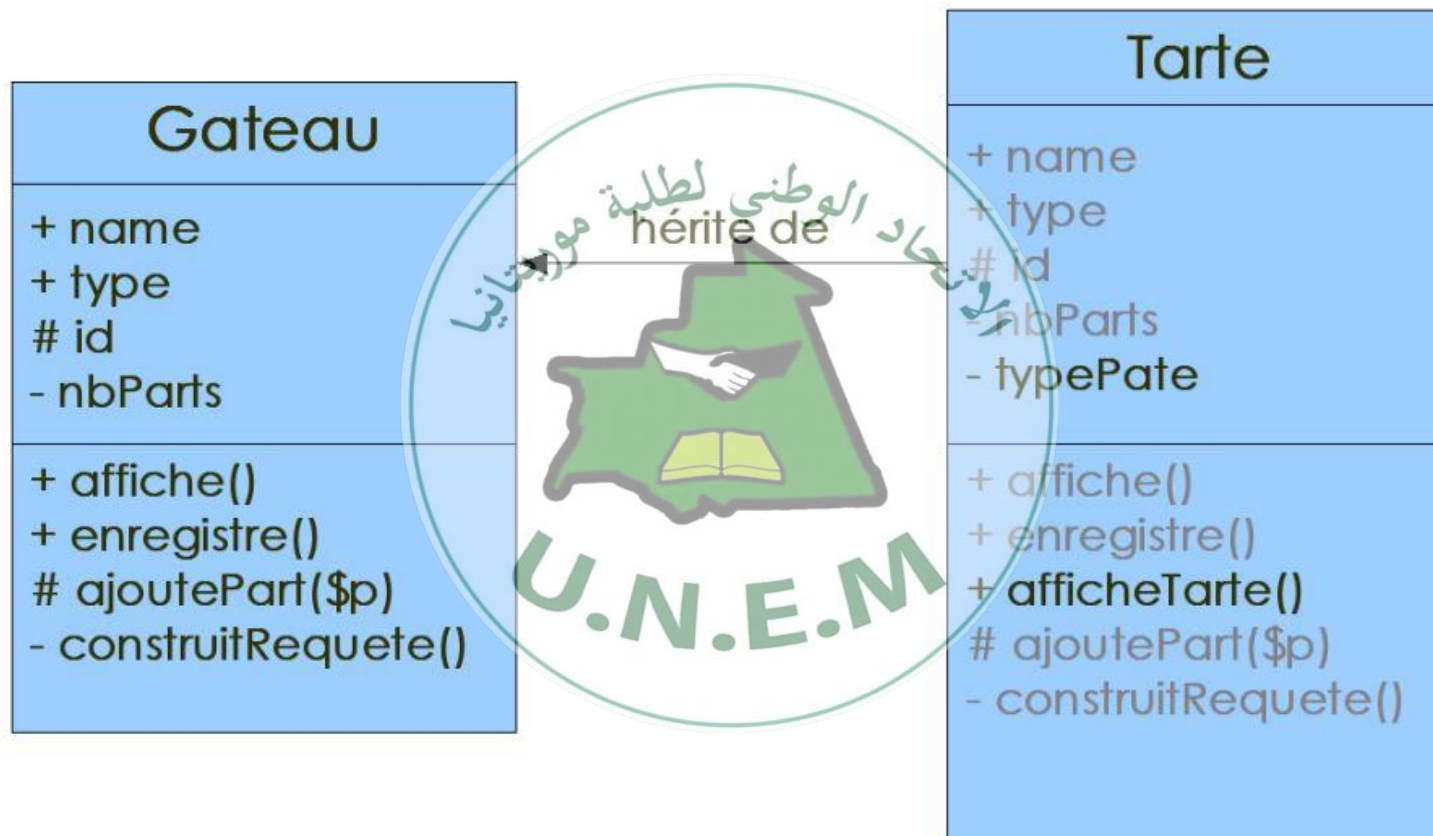


L'héritage

- L'héritage permet de hiérarchiser les classes.
- Les classes mères sont génériques, elles mettent en place des fonctionnalités communes à une ou plusieurs classes.
- Les classes filles sont spécifiques, elles héritent des fonctionnalités des classes mères.



L'héritage (UML)



classe mère

classe fille

Exemple Héritage (1/2)

```
<?php

class User // Une classe Utilisateur
{
    private $name;

    public function setName($name)
    {
        $this->name = $name;
    }

    public function getName()
    {
        return ucfirst($this->name);
    }
}
```



Héritage (2/2)

```
// Une classe 'Administrator' qui hérite de 'User'
class Administrator extends User
{
    private $level = 'administrator';
}

// Administrator hérite de User donc on peut
// s'en servir comme d'un User

$admin = new Administrator();
$admin->setName('jean');
echo $admin->getName();
```



Retour sur notre exemple Animal

- Quelques notions essentielles



Les constantes : utilité

- Les constantes de classe permettent d'éviter tout code *muet*. Voici un code muet :
- `<?php`
- `$animalO = new Animal(50);`
- Pourquoi est-il muet ? Tout simplement parce qu'on ne sait pas à quoi « 50 » correspond.
Qu'est-ce que cela veut dire ? Étant donné que je viens de réaliser le script, je sais que ce « 50 » correspond à la puissance d'un Animal.



Constante c'est quoi

Une constante est une sorte d'attribut appartenant à la classe dont la valeur ne change jamais. Pour déclarer une constante, vous devez faire précéder son nom du mot-clé `const`. Faites bien attention, une constante ne prend pas de « \$ » devant son nom ! Voici donc comment créer une constante :



Hypothèse sur le setter de la puissance

- Supposons que la puissance peut prendre 3 valeurs possibles :
- 20, qui veut dire que l'Animal aura une faible puissance ;
- 50, qui veut dire que l'Animal aura une puissance moyenne ;
- 80, qui veut dire que l'Animal sera très fort.
- Au lieu de passer ces valeurs telles quelles, on va plutôt passer une **constante** au constructeur. Ainsi, quand on lira le code, on devinera facilement que l'on passe une puissance moyenne au constructeur. C'est bien plus facile à comprendre qu'un nombre quelconque.



- <?php
- class Animal{
- private \$puissance;
- private \$localisation;
- private \$experience;
- private \$dommages;
- // Déclarations des constantes en rapport avec la puissance.
- const PUISSANCE_PETITE = 20;
- const PUISSANCE_MOYENNE = 50;
- const PUISSANCE_GRANDE = 80;
- public function deplacer(){ }
- public function taper(){ }
- public function gagnerExperience(){ }}



Exemple de setter puissance

- public function setPuissance(\$puissance)
- {
- // On vérifie qu'on nous donne bien soit une «
PUISSANCE_PETITE », soit une «
PUISSANCE_MOYENNE »,
- // soit une « PUISSANCE_Grande ».
- if (in_array(\$puissance, [self::PUISSANCE_PETITE,
- self::PUISSANCE_MOYENNE,
- self::PUISSANCE_Grande]))
- {
- \$this->puissance = \$puissance;



- Notez qu'ici les noms de constantes sont en majuscules : c'est encore et toujours une convention de dénomination.
- **Les attributs et méthodes statiques**
- Comme je l'ai brièvement dit dans l'introduction, les méthodes statiques sont des méthodes qui sont faites pour agir sur une classe et non sur un objet. Par conséquent, je ne veux voir aucun \$this dans la méthode ! En effet, la méthode n'étant appelée sur aucun objet, il serait illogique que cette variable existe. Souvenez-vous : \$this est un paramètre implicite. Cela n'est vrai que pour les méthodes appelées sur un objet !



- Même si la méthode est dite « statique », il est possible de l'appeler depuis un objet (\$obj->methode Statique()), mais, même dans ce contexte, la variable \$this ne sera toujours pas passée !
- Pour déclarer une méthode statique, vous devez faire précéder le mot-clé function du mot-clé static après le type de visibilité.



Les attributs statiques

- Le principe est le même, c'est-à-dire qu'un attribut statique appartient à la classe et non à un objet. Ainsi, tous les objets auront accès à cet attribut et cet attribut aura la même valeur pour tous les objets.
- La déclaration d'un attribut statique se fait en faisant précéder son nom du mot-clé static, comme ceci :



Voici la façon avec laquelle on declare l'attribut static :

```
// Variable statique PRIVÉE.  
private static $_texteADire = 'Je vous encourage!';  
Pour l'appeler:
```



`echo self::$_texteADire;`

Regardons l'exemple animal6 pour plus de détails

A récapituler

- L'opérateur « -> » permet d'accéder à un élément de tel objet, tandis que l'opérateur « :: » permet d'accéder à un élément de telle classe.
- Au sein d'une méthode, on accède à l'objet grâce à la pseudo-variable \$this, tandis qu'on accède à la classe grâce au mot-clé self.
- Les attributs et méthodes statiques ainsi que les constantes de classe sont des éléments propres à la classe.



Les constantes de classe sont utiles pour éviter d'avoir un code muet.

- Un objet se crée grâce à l'opérateur new.
- L'accès à un attribut ou à une méthode d'un objet se fait grâce à l'opérateur «->».
- Pour lire ou modifier un attribut, on utilise des *getters* et des *setters*.
- Le constructeur d'une classe a pour rôle principal d'initialiser l'objet en cours de création, c'est-à-dire d'initialiser la valeur des attributs (soit en assignant directement des valeurs spécifiques, soit en appelant diverses méthodes).
- Les classes peuvent être chargées dynamiquement (c'est-à-dire sans avoir explicitement inclus le fichier la déclarant) grâce à l'auto-chargement de classe (utilisation de `_autoload_register`).



Exercice

- Ecrire, dans un fichier Point.php, une classe pour représenter des points en 2 dimensions. Elle comprendra au moins 1 constructeur ainsi qu'une méthode pour calculer la distance d'un point à un autre



Exercice

Pour mettre en pratique la notion d'orienté objets, Vous allez créer une class « form » permettant la génération des champs du formulaire suivants :

- Le champ input
- Le champ select
- Le champ submit

Pour exploiter notre nouvelle class, vous allez créer un fichier nommé « index.php » contenant un formulaire de 4 champs « email », « tel », nom et sexe, les données



ies seront stocké dans la table personne de la base de donné BD2.